«ЖИТОМИРСЬКА ПОЛІТЕХНІКА»
ФАКУЛЬТЕТ ІНФОРМАЦІЙНО-КОМП'ЮТЕРНИХ ТЕХНОЛОГІЙ
КАФЕДРА КОМП'ЮТЕРНОЇ ІНЖЕНЕРІЇ

ЗВІТ

з лабораторної роботи № 2

з дисципліни «Програмування»

Тема: Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Варіант № 1

Виконав:

Студент 1-го курсу, групи КІ-26-1

Спеціальність: 123 «Комп'ютерна інженерія»

Артем

Перевірив:

Викладач кафедри

комп'ютерної інженерії

Лисенко Максим Сергійович

Житомир - 2026

2.1 Мета роботи

- Набути практичних навичок проектування комбінованих алгоритмів із використанням операторів розгалуження (`if-else`) та циклічних конструкцій.
- Дослідити алгоритмічні особливості та практичну реалізацію циклів із передумовою (`while`).
- Опанувати чисельні методи наближеного обчислення функцій за допомогою степеневих рядів та рекурентних співвідношень.
- Закріпити навички системного контролю версій Git: робота з репозиторієм, семантичні коміти за стандартом Conventional Commits.
- Опанувати академічні стандарти підготовки технічної документації шляхом формування обов'язкової пояснювальної записки у форматі PDF.

Індивідуальне завдання (Варіант № 1)

> **Розрахунок варіанта за порядковим номером у списку групи (N = 31):**
>
> Варіант = `(31 - 1) mod 15 + 1 = 30 mod 15 + 1 = 0 + 1 = 1`.

Завдання 1. Табулювання кусочно-неперервної функції (task1.c)

Розробити програму мовою C для табулювання кусочно-неперервної функції *Y(X)* на заданому проміжку `[X_start, X_end]` із кроком `dX`.

Математичний вираз кусочної функції для Варіанта 1:

$$Y(X) = \begin{cases} -X, & \text{якщо } X \le 0; \\ X^2, & \text{якщо } 0 < X \le 2; \\ 4.0, & \text{якщо } X > 2 \end{cases}$$

Завдання 2. Наближене обчислення функцій чисельним рядом (task2.c)

Розробити програму мовою C для наближеного обчислення функції e^X (експонента) за допомогою розкладу в нескінченний степеневий ряд Маклорена із заданою інженерною точністю ε.

3.1. Аналіз ОДЗ та неперервності (Завдання 1)

Кусочно-неперервна функція для Варіанта 1 задана виразом:

$$Y(X) = \{ -X\ (X \le 0),\quad X^2\ (0 < X \le 2),\quad 4.0\ (X > 2) \}$$

Проведемо математичний аналіз області допустимих значень (ОДЗ) та поведінки функції за інтервалами:

Інтервал 1 (X ≤ 0)

- Вираз `Y(X) = -X` є лінійною функцією.
- Не містить операцій ділення на вирази зі змінною чи добування кореня з від'ємних чисел.
- Функція визначена та неперервна для всіх $X \in (-\infty; 0]$.

Інтервал 2 (0 < X ≤ 2)

- Вираз `Y(X) = X^2` є квадратичною (степеневою) функцією.
- Визначений на всій числовій осі, зокрема на заданому півінтервалі $(0; 2]$.
- Точок розриву або невизначеності на цьому інтервалі немає.

Інтервал 3 (X > 2)

- Вираз `Y(X) = 4.0` є сталою (константною) функцією.
- Визначений для всіх $X > 2$ без жодних обмежень.

> **Висновок за Завданням 1:** Функція не має точок розриву другого роду (деталізованих асимптот чи ділення на нуль), граничні переходи в точках стику $X = 0$ та $X = 2$ забезпечують неперервність у цілому.

3.2. Аналітичний вивід рекурентного співвідношення (Завдання 2)

Для наближеного обчислення функції e^X використовується розклад у ряд Маклорена (ряд Тейлора в точці 0):

$$e^X = \sum_{n=0}^{\infty} (X^n / n!) = 1 + (X / 1!) + (X^2 / 2!) + (X^3 / 3!) + \dots$$

Область збіжності даного ряду становить всю числову вісь $(-\infty < X < \infty)$, що гарантує збіжність для будь-якого дійсного аргументу X.

Виведення рекурентної формули

Запишемо загальний n-й член ряду a_n та попередній $(n-1)$-й член a_{n-1}:

$a_n = X^n / n!$

$a_{n-1} = X^{n-1} / (n-1)!$

Знайдемо коефіцієнт переходу $R(n, X) = a_n / a_{n-1}$:

$$R(n, X) = (X^n / n!) / (X^{n-1} / (n-1)!) = (X^n / X^{n-1}) \cdot ((n-1)! / n!)$$

Спростимо кожен із двох множників:

- Степінь: $X^n / X^{n-1} = X^{n-(n-1)} = X$
- Факторіал: $(n-1)! / n! = (n-1)! / ((n-1)! \cdot n) = 1 / n$

Підставляючи результати, отримуємо рекурентне співвідношення:

$$a_0 = 1.0, \quad a_n = a_{n-1} \cdot (X / n) \quad (\text{для } n \geq 1)$$

4.1. Логіка роботи та захисні заходи у task1.c

Валідація вхідних даних: Програма зчитує *X_start*, *X_end* та *dX*. Якщо введений крок $dX \leq 0$ або початкове значення більше за кінцеве ($X_start > X_end$), виводиться повідомлення у потік `stderr`, і програма завершується з кодом `1`.

Цикл табулювання: Використовується цикл `while` з кроком *dX*. Для коректної обробки кінцевої точки з урахуванням похибок IEEE 754 умова завершення задається як `x <= x_end + eps`, де `eps = dX / 1000.0`.

Каскадне розгалуження: На кожній ітерації перевіряється умова для вибору формули:

- Якщо $X \leq 0$: обчислюється $Y = -X$.
- Якщо $0 < X \leq 2$: обчислюється $Y = X^2$.
- Якщо $X > 2$: обчислюється $Y = 4.0$.

4.2. Логіка роботи та захисні заходи у task2.c

Обчислення ряду: Програма реалізує накопичення суми експоненти за допомогою рекурентного множника без виклику важких функцій `pow()` чи обчислення факторіала на кожній ітерації.

Захисні заходи (Timeout Guard): Для запобігання нескінченним циклам у разі збурень введено обмеження максимальної кількості ітерацій `MAX_ITERATIONS = 100000`. Точність перевіряється за модулем члена ряду: `fabs(term) < epsilon`.

Верифікація: Отриманий результат порівнюється зі стандартною бібліотечною функцією `exp(x)` з заголовка `<math.h>` з виведенням абсолютної похибки.

5.1. Повний лістинг файлу task1.c

```c
#include <stdio.h>
#include <math.h>

int main() {
    double x_start, x_end, dx;

    printf("Enter x_start: ");
    if (scanf("%lf", &x_start) != 1) return 1;

    printf("Enter x_end: ");
    if (scanf("%lf", &x_end) != 1) return 1;

    printf("Enter step (dx): ");
    if (scanf("%lf", &dx) != 1) return 1;

    if (x_start > x_end || dx <= 0) {
        fprintf(stderr, "Error: invalid parameters (x_start must be <= x_end, and dx > 0).\n");
        return 1;
    }

    printf("\n-----------------------------------\n");
    printf("|      X      |       Y(X)         |\n");
    printf("-----------------------------------\n");

    for (double x = x_start; x <= x_end + 1e-9; x += dx) {
        double y;

        if (x <= 0.0) {
            y = -x;
        } else if (x > 0.0 && x <= 2.0) {
            y = x * x;
        } else {
            y = 4.0;
        }

        printf("| %11.4lf | %16.4lf |\n", x, y);
    }

    printf("-----------------------------------\n");
    return 0;
}
```

5.2. Повний лістинг файлу task2.c

```c
#include <stdio.h>
#include <math.h>

int main() {
    double x, epsilon;

    printf("Enter argument X: ");
    if (scanf("%lf", &x) != 1) return 1;

    printf("Enter desired precision epsilon (e.g., 0.00001): ");
    if (scanf("%lf", &epsilon) != 1) return 1;

    if (epsilon <= 0) {
        fprintf(stderr, "Error: precision must be a positive number.\n");
        return 1;
    }

    double sum = 1.0;       // Initial sum (n=0, a_0 = 1)
    double term = 1.0;      // Current series term a_0
    int n = 1;
    int max_iterations = 100000; // Timeout Guard
    int converged = 0;

    while (n <= max_iterations) {
        // Recurrent relation: a_n = a_{n-1} * (X / n)
        term = term * (x / n);
        sum += term;

        if (fabs(term) < epsilon) {
            converged = 1;
            break;
        }
        n++;
    }

    if (!converged) {
        printf("Warning: iteration limit reached (%d), precision not guaranteed!\n", max_iterations);
    }

    double exact_value = exp(x);
    double absolute_error = fabs(sum - exact_value);

    printf("\n--- Results for e^(%.2lf) ---\n", x);
    printf("Computed series sum:     %.7lf\n", sum);
    printf("Standard library (exp):  %.7lf\n", exact_value);
    printf("Absolute error:          %.7le\n", absolute_error);
    printf("Iterations spent:        %d\n", n);

    return 0;
}
```

6.1. Результати роботи програми task1.c

При запуску програми табулювання для проміжку від `-2.0` до `4.0` із кроком `1.0`:

```
-----------------------------------
|      X      |       Y(X)        |
-----------------------------------
|     -2.0000 |         2.0000    |
|     -1.0000 |         1.0000    |
|      0.0000 |         0.0000    |
|      1.0000 |         1.0000    |
|      2.0000 |         4.0000    |
|      3.0000 |         4.0000    |
|      4.0000 |         4.0000    |
-----------------------------------
```

6.2. Результати роботи програми task2.c

При введенні аргументу $X = 1.0$ та точності $\varepsilon = 0.00001$:

```
--- Results for e^(1.00) ---
Computed series sum:     2.7182788
Standard library (exp):  2.7182818
Absolute error:          3.0586178e-06
Iterations spent:        8
```

7. ВИСНОВКИ

У ході виконання лабораторної роботи № 2 було здобуто практичні навички розробки алгоритмів із розгалуженими та циклічними процесами мовою C, а також опановано чисельні методи наближеного обчислення функцій за допомогою степеневих рядів.

Основні результати роботи:

- **Табулювання кусочно-неперервної функції (task1.c):** Реалізовано програму для табулювання функції $Y(X)$ відповідно до Варіанта 1. Забезпечено коректне обчислення на всіх трьох інтервалах з використанням розгалужених конструкцій та захисних перевірок вхідних параметрів.
- **Наближене обчислення e^X чисельним рядом (task2.c):** Опановано чисельний метод обчислення значення експоненти за допомогою розкладу в ряд Маклорена. Завдяки аналітичному виводу рекурентного співвідношення $a_n = a_{n-1} \cdot (X / n)$ досягнуто високої обчислювальної ефективності без викликів важких математичних функцій.
- **Захисні механізми:** Впроваджено валідацію точності $\varepsilon > 0$ та апаратний захист від зациклення (Timeout Guard до 100 000 ітерацій). Тестування показало швидку збіжність ряду (всього 8 ітерацій для $X = 1.0$ до точності 10^{-5}).
- **Практичні навички:** Закріплено навички роботи з системою контролю версій Git, оформлення комітів за стандартом Conventional Commits та створення якісної інженерної документації.

> **Мету лабораторної роботи досягнуто в повному обсязі.**